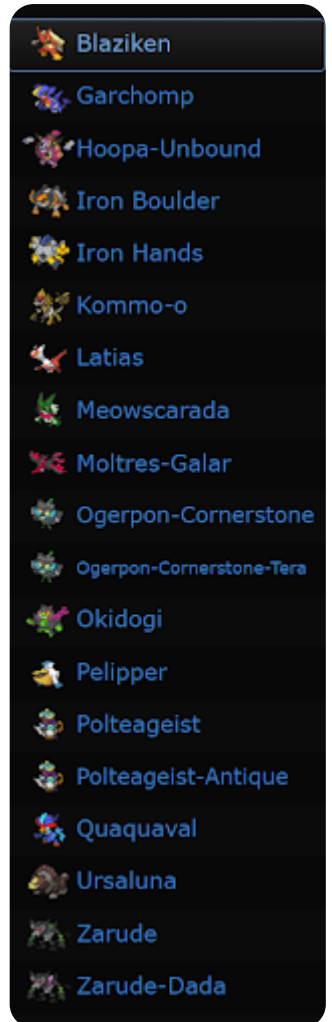


2026-05-02 22:11:09

You Asked 



can you make a python list out of the following:

 Gemini 3 Flash

Certainly! Here is the Python list extracted from the image, formatted as strings:

python

```
pokemon_list = [  
    "Blaziken",  
    "Garchomp",  
    "Hoopa-Unbound",  
    "Iron Boulder",  
    "Iron Hands",  
    "Kommo-o",  
    "Latias",  
    "Meowscarada",  
    "Moltres-Galar",  
    "Ogerpon-Cornerstone",  
    "Ogerpon-Cornerstone-Tera",  
    "Okidogi",  
    "Pelipper",  
    "Polteageist",  
    "Polteageist-Antique",  
    "Quaquaval",  
    "Ursaluna",  
    "Zarude",  
    "Zarude-Dada",  
]
```

```
"Meowscarada",  
"Moltres-Galar",  
"Ogerpon-Cornerstone",  
"Ogerpon-Cornerstone-Tera",  
"Okidogi",  
"Pelipper",  
"Polteageist",  
"Polteageist-Antique",  
"Quaquaval",  
"Ursaluna",  
"Zarude",  
"Zarude-Dada"
```

]

2026-05-02 22:14:09 You Asked 



-  Ammarouge
-  Blastoise
-  Enamorus-Therian
-  Gyarados
-  Hawlucha
-  Haxorus
-  Iron Leaves
-  Lilligant-Hisui
-  Mienshao
-  Oricorio-Pom-Pom
-  Salamence
-  Serperior
-  Thundurus
-  Volcanion
-  Zoroark-Hisui

same for the following:

Got it! Here is the Python list for the second set of Pokémon:

```
python
```

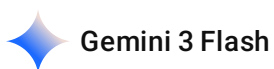
```
pokemon_list_2 = [  
    "Armarouge",  
    "Blastoise",  
    "Enamorus-Therian",  
    "Gyarados",  
    "Hawlucha",  
    "Haxorus",  
    "Iron Leaves",  
    "Lilligant-Hisui",  
    "Mienshao",  
    "Oricorio-Pom-Pom",  
    "Salamence",  
    "Serperior",  
    "Thundurus",  
    "Volcanion",  
    "Zoroark-Hisui"  
]
```

2026-05-02 22:15:43

You Asked 



once more:



Here is the third list for you:

```
python
```

```
pokemon_list_3 = [  
    "Azelf",  
    "Cetitan",  
    "Cloyster",  
    "Cresselia",  
    "Deoxys-Defense",  
    "Feraligatr",  
    "Iron Thorns",  
    "Lucario",  
    "Lycanroc-Dusk",  
    "Oricorio-Sensu",  
    "Regidrago",  
    "Thwackey"]
```

]

2026-05-02 22:19:12 You Asked 



Alcremie



Bellossom



Electrode-Hisui



Emboar



Kingdra



Oricorio

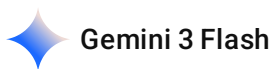


Oricorio-Pa'u



Venomoth

one last time:



Here is the final list for you:

```
python
```

```
pokemon_list_4 = [  
    "Alcremie",  
    "Bellossom",  
    "Electrode-Hisui",  
    "Emboar",  
    "Kingdra",  
    "Oricorio",  
    "Oricorio-Pa'u",  
    "Venomoth"
```

convert these two into python arrays, only have the names as strings for the smogon ban list. For the manual cleanup have each element be a tuple with the name as a string in the first element and the part of the "-" as the second: Smogon Ban

Arceus-Fire
Arceus-Flying
Arceus-Ghost
Arceus-Steel
Arceus-Water
Lunala
Shaymin-Sky
Zekrom

Manual Clean up

Wo-Chien - PU
Gothorita - remove
Mamoswine - UU
Trapinch - remove
Gothita - remove
Scrafty - PU
Cottonee - remove
Comfey - RU
Riolu - remove
Murkrow - remove
Jumpluff - ZU
Sandshrew - remove
Heracross - PU
Decidueye - NU
Impidimp - remove
Slowbro-Galar - PU

Here are the two Python lists based on your requirements:

python

```
# Smogon Ban List
smogon_ban = [
    "Arceus-Fire",
    "Arceus-Flying",
    "Arceus-Ghost",
    "Arceus-Steel",
    "Arceus-Water",
    "Lunala",
    "Shaymin-Sky",
    "Zekrom"
]

# Manual Clean up List
manual_cleanup = [
    ("Wo-Chien", "PU"),
    ("Gothorita", "remove"),
    ("Mamoswine", "UU"),
    ("Trapinch", "remove"),
    ("Gothita", "remove"),
    ("Scrafty", "PU"),
    ("Cottonee", "remove"),
    ("Comfey", "RU"),
    ("Riolu", "remove"),
    ("Murkrow", "remove"),
    ("Jumpluff", "ZU"),
    ("Sandshrew", "remove"),
    ("Heracross", "PU"),
    ("Decidueye", "NU"),
    ("Impidimp", "remove"),
    ("Slowbro-Galar", "PU")
]
```

is there a way to make the variable `ubers_pkmn` name dynamic i.e. make it `tier.lower + _pkmn`?

Gemini 3 Flash

In Python, while you *can* technically create dynamic variable names using `globals()` or `locals()`, it's generally considered poor practice because it makes the code hard to debug and "magic" to read.

The "Pythonic" way to handle this is by using a **dictionary**. This allows you to map a tier name to its list of Pokémon while keeping the naming logic clean.

The Dictionary Approach (Recommended)

You can store your data in a dictionary and use an f-string to define the keys dynamically.

```
python

tier = "Ubers"
pokemon_data = {
    # This creates the key 'ubers_pkmn'
    f"{tier.lower()}_pkmn": [
        "Arceus-Fire", "Arceus-Flying", "Arceus-Ghost",
        "Arceus-Steel", "Arceus-Water", "Lunala",
        "Shaymin-Sky", "Zekrom"
    ]
}

# Accessing it
print(pokemon_data["ubers_pkmn"])
```

The "Dynamic Variable" Way (The Hack)

If you absolutely must have a standalone variable named `ubers_pkmn` without using a dictionary, you can use the `globals()` function. This injects the name directly into the script's global namespace.

```
python

tier = "Ubers"
var_name = f"{tier.lower()}_pkmn"

# Assigning the list to the dynamic name
globals()[var_name] = ["Arceus-Fire", "Arceus-Flying", "Zekrom"]
```



```
# Now you can call it directly
print(ubers_pkmn)
```

Why go with the Dictionary?

1. **Safety:** `globals()` can accidentally overwrite existing built-in functions or other variables.
2. **Organization:** If you have multiple tiers (UU, RU, NU), you can keep them all in one `tiers_data` dictionary rather than having 10+ loose variables floating around.
3. **Iteration:** It's much easier to loop through a dictionary than to search the entire global memory for variables ending in `_pkmn`.

2026-05-02 22:39:02 You Asked 

```
sorry this is code I was asking about: for pkmn, tier in
manual_cleanup:
    del uubers_pkmn[pkmn]
    if not(tier == "remove"):
        ubers_pkmn[pkmn] = tier
```

Gemini 3 Flash

I see what you're doing—you're trying to move Pokémon out of one list and into another (or update their tiering) based on that `manual_cleanup` list.

If you want the destination variable to be determined by the `tier` name dynamically within that loop, the safest and most efficient way is to use a **dictionary of lists** (or dictionaries) to hold your tiers.

The Recommended Way (Using a Tiers Dictionary)

Instead of having independent variables like `uu_pkmn`, `ru_pkmn`, etc., put them into one container. This makes your loop very clean:

```
python

# Setup your data structure
pkmn_by_tier = {
    "uu": {"Mamoswine": "UU", "Other": "UU"},
    "pu": {},
    "ru": {},
    "nu": {},
    "zu": {}
```

```

}

for pkmn, tier in manual_cleanup:
    # 1. Logic to remove from current (e.g., UU)
    # We use .pop(key, None) so it doesn't crash if the pkmn isn't there
    pkmn_by_tier["uu"].pop(pkmn, None)

    # 2. Add to the new tier if it's not "remove"
    if tier != "remove":
        target_tier = tier.lower()
        # Add to the specific tier dictionary
        pkmn_by_tier[target_tier][pkmn] = tier

# Now you access them like this:
print(pkmn_by_tier["pu"])

```

The "Dynamic Variable" Way (If you must)

If you already have variables like `pu_pkmn` and `ru_pkmn` defined in your script and you don't want to change your whole structure, you can use `globals()` to fetch the variable based on the string name.

Warning: This is a bit "hacky" and can be brittle if you mistype a tier name.

python

```

for pkmn, tier in manual_cleanup:
    # Remove from your primary list (assuming it's uubers_pkmn)
    if pkmn in uubers_pkmn:
        del uubers_pkmn[pkmn]

    if tier != "remove":
        # Construct the variable name string, e.g., "pu_pkmn"
        var_name = f"{tier.lower()}_pkmn"

        # Access the variable from the global scope and update it
        # This assumes pu_pkmn, ru_pkmn, etc., are already defined as dictionaries
        globals()[var_name][pkmn] = tier

```

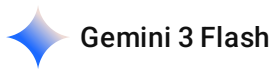
Why the Dictionary is better:

In your code snippet, you were using `uubers_pkmn[pkmn] = tier`. If `tier` is "PU", it should probably go into `pu_pkmn`, not `uubers_pkmn`. Using a dictionary (`pkmn_by_tier[tier.lower()]`) handles that routing automatically.

Does your current project have all the tier lists (UU, RU, NU) already initialized as variables?



I have dictionaries for the following pokemon tiers: OU, UU, RU, NU, PU and ZU, I want piece of code that checks if there are any duplicate pokemon in my dictionaries and if so print them



The most efficient way to check for duplicates across multiple dictionaries is to use a `defaultdict` to track which tiers each Pokémon appears in.

Since you have six different tier variables, we'll put them into a list or dictionary to iterate through them easily.

The Code

python

```
from collections import defaultdict

# 1. Put your dictionaries into a master dictionary for easy iteration
tier_data = {
    "OU": ou_pkmn,
    "UU": uu_pkmn,
    "RU": ru_pkmn,
    "NU": nu_pkmn,
    "PU": pu_pkmn,
    "ZU": zu_pkmn
}

# 2. Map every pokemon to the list of tiers it is found in
pkmn_tracker = defaultdict(list)

for tier_name, pkmn_dict in tier_data.items():
    for pokemon in pkmn_dict:
        pkmn_tracker[pokemon].append(tier_name)

# 3. Print only the duplicates
duplicates_found = False
print("--- Checking for Duplicates ---")

for pokemon, tiers in pkmn_tracker.items():
```

```

if len(tiers) > 1:
    duplicates_found = True
    print(f"⚠️ DUPLICATE: {pokemon} is in {'', '.join(tiers)}")

if not duplicates_found:
    print("✅ No duplicates found! All tiers are unique.")

```

Why this works:

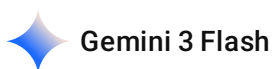
1. **defaultdict(list)** : This creates a dictionary where every new entry automatically has an empty list. Instead of checking `if pokemon in tracker` , we just `.append()` .
2. **tier_data Mapping**: By grouping your variables into one dictionary, you can easily add or remove tiers (like `ubers_pkmn`) in the future without changing the logic.
3. **len(tiers) > 1** : This specifically filters for Pokémon that exist in more than one dictionary.

Quick Tip:

If you want to **fix** them automatically while you run the code, you can use the loop to keep the Pokémon in the *highest* tier (the first one it appears in) and delete it from the lower ones. Would you like a snippet for an auto-cleaner as well?

2026-05-02 23:05:02 You Asked 

take this edit it so that not only is the sum printed but also the percentage contributed by each tier: # Dataset size
 print(len(ubers_pkmn) + len(uubers_pkmn) + len(ou_pkmn) +
 len(uu_pkmn) + len(ru_pkmn) + len(nu_pkmn) + len(pu_pkmn) +
 len(zu_pkmn))



To make this clean and readable, it's best to store the counts in a dictionary first. This avoids repeating the `len()` calls and makes calculating the percentages much easier.

Here is the updated code:

```

python

# 1. Store counts in a dictionary for easy calculation
tier_counts = {
    "Ubers": len(ubers_pkmn),

```

```

    "UUBers": len(uubers_pkmn),
    "OU": len(ou_pkmn),
    "UU": len(uu_pkmn),
    "RU": len(ru_pkmn),
    "NU": len(nu_pkmn),
    "PU": len(pu_pkmn),
    "ZU": len(zu_pkmn)
}

total_pkmn = sum(tier_counts.values())

# 2. Print the individual breakdown and percentages
print(f"{'Tier':<10} | {'Count':<6} | {'Percentage'}")
print("-" * 30)

for tier, count in tier_counts.items():
    percentage = (count / total_pkmn) * 100 if total_pkmn > 0 else 0
    print(f"{'Tier':<10} | {'Count':<6} | {'percentage:.2f}%")

print("-" * 30)
print(f"{'TOTAL':<10} | {'total_pkmn':<6} | 100.00%")

```

What's happening here:

- **sum(tier_counts.values())** : Automatically adds up all your lengths so you don't have to manually write a long addition string.
- **F-String Formatting:** I used `{tier:<10}` to align the text into columns so it looks like a clean table in your console.
- **{percentage:.2f}%** : This rounds the percentage to two decimal places (e.g., **12.54%**) so it's not a long, messy float.
- **Safety Check:** Added a check (`if total_pkmn > 0`) just in case you run this with empty lists, preventing a "Division by Zero" error.